

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	COMPUTACIÓN GRAFICA, VISIÓN COMPUTACIONAL Y MULTIMEDIA (E)				
TÍTULO DE LA PRÁCTICA:	<i>Transformaciones Geometricas en 2D</i>				
NÚMERO DE PRÁCTICA:	<i>02</i>	AÑO LECTIVO:	<i>5to</i>	NRO. SEMESTRE:	<i>NOVENO</i>
FECHA DE PRESENTACIÓN	<i>dd/mm/aaaa</i>	HORA DE PRESENTACIÓN	<i>hh/mm/ss</i>		
INTEGRANTE (s) <i>Sencia Ale Bryan Daniel</i>				NOTA (0-20)	
DOCENTE(s): MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.					

RESULTADOS Y PRUEBAS
Link del video: Juego EscapeCity protagonizado por Kale, (el audio esta bajo) I. DESGLOSE DEL PROYECTO ESCAPECITY DE KALE a. IMPLEMENTACIÓN: ENTIDAD PRINCIPAL (KALE) Cuando desarrollamos el personaje principal hemos aplicado transformaciones geométricas para gestionar el movimiento, la orientación y el cambio de estado biomórfico (humano-lobo). Aquí es donde le damos vida al personaje usando pura matemática afín: $P' = AP + W$ i. Traslación (W): Movimiento y Salto La traslación es básicamente mover un punto a otro sumándole un vector W. En el código, no teletransportamos al personaje, sino que calculamos cuánta "fuerza" de movimiento (W) le aplicamos en cada frame. Análisis del Código: Para el movimiento horizontal, multiplicamos el input del jugador por una constante de velocidad, mientras que para el salto aplicamos un impulso vertical instantáneo. <pre>// Traslación Horizontal (FixedUpdate) Rigidbody2D.linearVelocity = new Vector2(Horizontal * Speed, Rigidbody2D.linearVelocity.y); // Traslación Vertical (Salto) Rigidbody2D.AddForce(Vector2.up * JumpForce, ForceMode2D.Impulse);</pre> Sustento Matemático: Cada frame, la nueva posición P' se obtiene sumando el desplazamiento acumulado al vector de posición original P: $P' = P + (V \cdot \Delta t)$

Donde V es el vector de velocidad resultante y Δt es el *Time.deltaTime*



Img. 01 - Línea roja del Debug.DrawRay

ii. Reflexión y Escalado (A): El "Giro" del Personaje

Para que Kale pueda mirar a la izquierda, no usamos rotación, sino una reflexión horizontal. Esto se logra mediante un escalado negativo en el eje X de la matriz A .

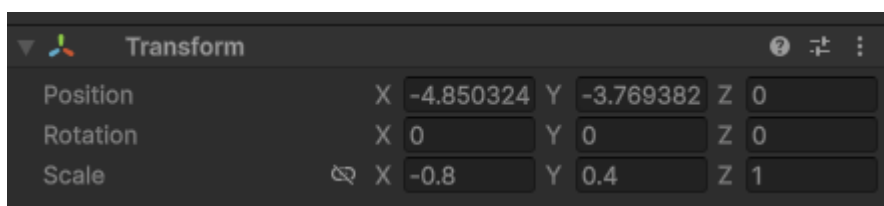
Análisis del Código:

Guardamos la escala original en *Start* y la invertimos multiplicando por -1 cuando el input es negativo.

```

if (Horizontal > 0) {
    transform.localScale = originalScale; // Escala original (0.8, 0.4, 1)
}
else if (Horizontal < 0) {
    // Inversión del eje X para aplicar reflexión
    transform.localScale = new Vector3(-originalScale.x,
    originalScale.y, originalScale.z);

```



Img. 02 - Reflexión del personaje y valores del inspector en ese momento.

Sustento Matemático:

Se aplica una transformación lineal donde la matriz A tiene un determinante negativo, invirtiendo el sentido de los vectores en el eje de las abscisas:

$$A = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}$$

iii. Transformación de Estado (Metamorfosis Humano-Lobo)

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

Esta mecánica es una composición de transformaciones que altera tanto la visibilidad como la geometría de colisión para adaptarse a la nueva forma.

- Traslación del Sensor (Raycast): El centro de masa cambia, por lo que el vector de traslación del rayo de suelo debe acortarse de 2.1 unidades a 0.8 unidades para que la detección sea precisa en modo lobo.
- Escalado de Colliders: Se desactivan y activan componentes para cambiar la "matriz de colisión" de una cápsula (humano) a una caja (lobo).

```
void ActualizarEstadoForma() {
    visualHumano.SetActive(!esLobo); // Escalado lógico (visibilidad)
    visualLobo.SetActive(esLobo);

    colliderHumano.enabled = !esLobo; // Cambio de geometría de colisión
    colliderLobo.enabled = esLobo;

    // Traslación dinámica del sensor de suelo
    RayDistance = esLobo ? rayDistLobo : rayDistHumano;
}
```



Img. 03 - Diferencia de colliders de ambos estados del personaje

iv. Composición de Rotación y Traslación: El Ataque (Onda)

En esta mecánica se demuestra cómo una rotación inicial altera el sentido de una traslación constante posterior. El proyectil no "sabe" si va a la izquierda o derecha; simplemente se traslada hacia su eje "derecho" local, el cual nosotros orientamos previamente.

Sustento Matemático.

Para que la onda viaje en sentido opuesto al eje $\$$ global, aplicamos una matriz de rotación A sobre el eje vertical Y .

- Rotación (A): Aplicamos $R_y(180^\circ)$. Esto invierte la dirección del vector unitario horizontal del objeto.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

- Composición: La posición final P' es el resultado de rotar el objeto y luego sumarle el vector de traslación W en cada frame:

$$P' = (R_y \cdot P) + W$$

Análisis del Código.

En el script KaleMovement.cs en la función *Attack()*, primero calculas la posición de aparición (spawn) y luego decides si el proyectil debe rotarse basándote en la escala de Kale:

```
void Attack() {
    //Traslación del punto de origen (W) para que no salga del centro
    de Kale
    float offsetY = esLobo ? 0.1f : 0.3f;
    Vector3 spawnPos = transform.position + new Vector3(direccion *
0.1f, offsetY, 0);
    //Instanciación con rotación de identidad (0,0,0)
    GameObject ondaInstancia = Instantiate(OndaPrefab, spawnPos,
Quaternion.identity);
    //Rotación Afín: Si miramos a la izquierda, rotamos 180 grados en Y
    if (direccion < 0) {
        ondaInstancia.transform.rotation = Quaternion.Euler(0, 180, 0);
    }
}
```



Img. 04 - Rotación de la onda de ataque dependiendo a donde se dirige Kale

b. IMPLEMENTACIÓN DE MECÁNICAS DE ENTORNO

En este apartado se describe la aplicación de transformaciones para la resolución de obstáculos, utilizando un sistema de comunicación entre objetos (Sensores y Actuadores)

i. Puente Levadizo: Rotación sobre Pivote

A diferencia de los objetos estándar que rotan sobre su centro, este puzzle requiere una rotación cinemática sobre un eje específico para habilitar el paso al jugador.

Sustento Matemático y Lógico

Traslación del Origen: Se modificó el centro de la matriz de transformación trasladando el Pivote al extremo inferior del sprite (y_{min}). Esto permite que la rotación afecte al objeto como una bisagra.

Cálculo Angular: Se utiliza la función *RotateTowards* para una transición suave, evitando el Gimbal Lock y manteniendo una velocidad angular (ω) constante:

$$q_t = RotateTowards(q_{current}, q_{target}, \omega, \Delta t)$$

Análisis del Código

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

La activación se realiza cuando la Onda (proyector) colisiona con el tronco, disparando el evento de rotación.

```
// Ejecución en PuenteControl.cs
void Update() {
    if (activado) {
        // Rotación progresiva hacia el ángulo meta (90 grados)
        transform.rotation =
Quaternion.RotateTowards(transform.rotation, rotacionMeta, velocidad *
Time.deltaTime);
    }
}
```



Img. 05 - Punto Pivote cambiado para lograr la rotación con respecto a ese eje.

ii. La Puerta de Contrapeso: Traslación Proporcional

Esta mecánica implementa una transformación de traslación (W) que depende directamente de una variable escalar: la sumatoria de masas detectadas.

Sustento Matemático: Interpolación Lineal (LERP)

La posición final de la puerta (P') se calcula mediante una combinación lineal de dos vectores de posición fijos ($P_{cerrada}$ y $P_{abierta}$):

$$P' = P_{cerrada} + r. (P_{abierta} - P_{cerrada})$$

Ratio de Carga (r): Es el factor de normalización (0 a 1) que determina qué tan abierta está la puerta según el peso depositado.

Análisis del Código

El sistema se divide en un sensor de presión y un actuador en la puerta.

// Cálculo del ratio r en PuertaContrapeso.cs

```
float ratio = (float)objetosEncima / pesoMaximoNecesario;
```

// Aplicación de la Traslación mediante Vector3.Lerp

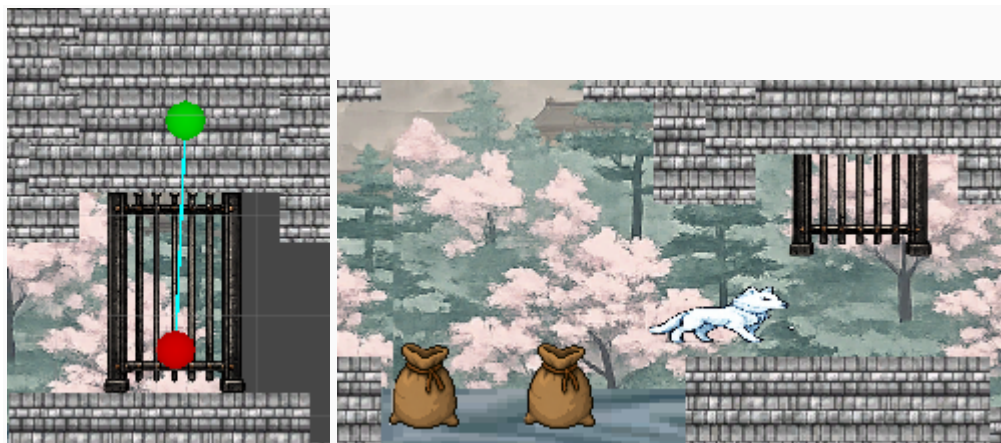
```
Vector3 destino = Vector3.Lerp(posCerrada.position,
posAbierta.position, Mathf.Clamp01(ratio));
transform.position = Vector3.Lerp(transform.position, destino,
velocidadSuave * Time.deltaTime);
```

Configuración Física de los Sacos

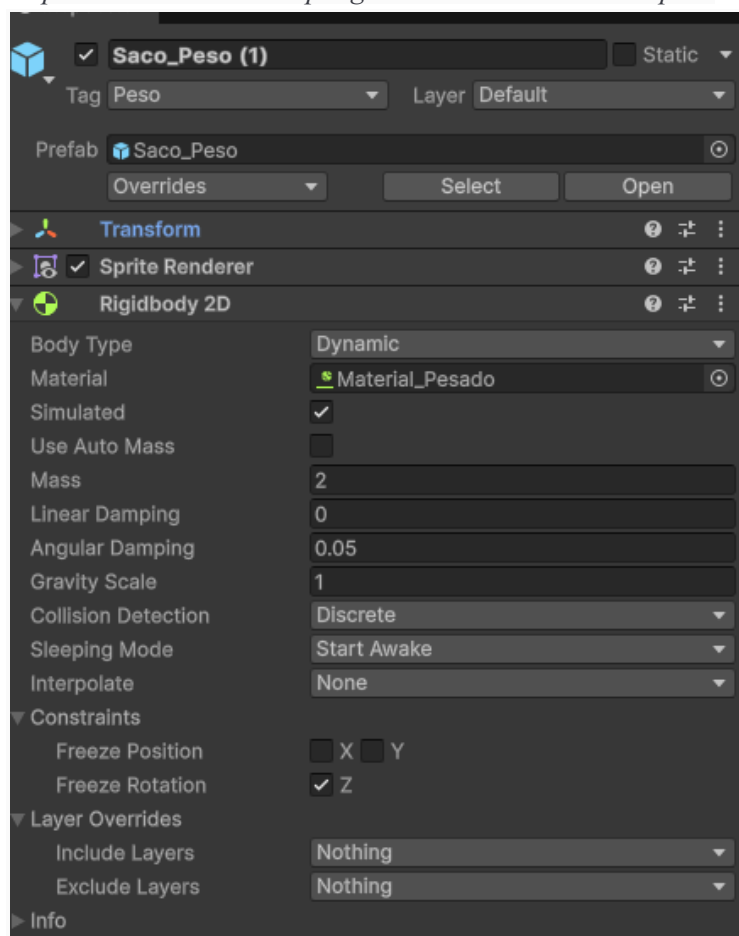
Para que la traslación sea estable, los objetos con el Tag "Peso" deben tener una configuración física específica:

- Constraints: Se activó *Freeze Rotation Z* para evitar que el saco ruede, asegurando que la superficie de contacto sea siempre plana.

- Material Físico: Se asignó un *Physics Material 2D* con Bounciness: 0 para evitar rebotes que generen falsas detecciones en el sensor.



Img. 06 - Se usó gizmos para ver el vector de desplazamiento, el recorrido que hará nuestra puerta se leve cuando pongamos los sacos de contrapeso



Img. 07 - Configuración del prefab sacos y sus principales características

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

c. TÉCNICAS MULTIMEDIA Y ENTORNO VISUAL

i. Efecto Parallax

Siguiendo lo requerido en la guía se implementó el efecto parallax

Efecto Parallax (Traslación Relativa)

El Parallax es una técnica de renderizado donde las capas de fondo se trasladan a distintas velocidades respecto a la cámara para simular profundidad en un entorno 2D.

Sustento Matemático:

Se aplica una traslación (W) calculada a partir del diferencial de movimiento de la cámara (ΔC_x) ponderado por un multiplicador de profundidad (k):

$$\Delta W_x = \Delta C_x \cdot k$$

Donde k es el *parallaxMultiplier*. Si k es cercano a 1, la capa se mueve casi a la par que la cámara (parece cercana); si es cercano a 0, la capa parece estar en el infinito.

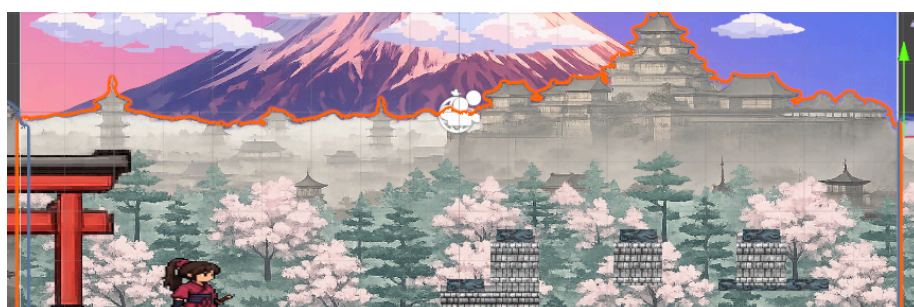
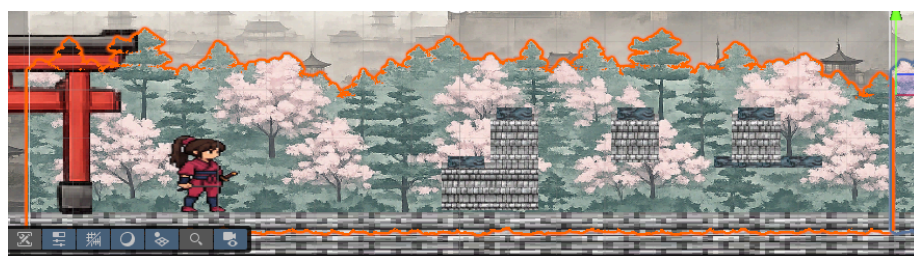
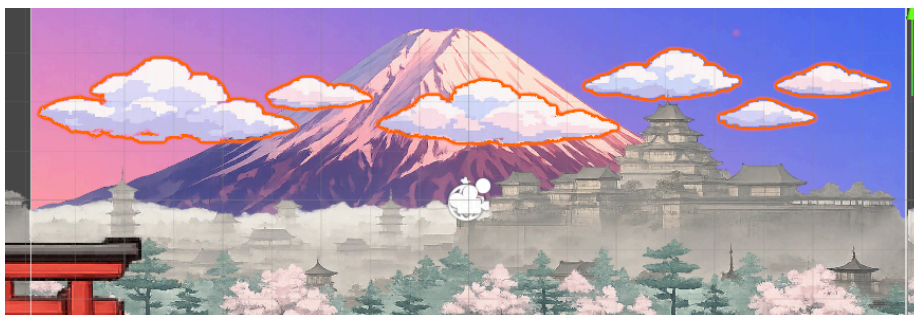
Análisis de la Implementación

El script *ParallaxEffect.cs* utiliza el método `transform.Translate` para aplicar el vector de traslación resultante en cada frame. Además, incluye una lógica de traslación cíclica para crear un fondo infinito:

```
void LateUpdate()
{
    // 1. Cálculo del vector de traslación relativo (W)
    float deltaX = (cameraTransform.position.x -
previousCameraPosition.x) * parallaxMultiplier;

    // Aplicación de la transformación P' = P + W
    transform.Translate(new Vector3(deltaX, 0, 0));

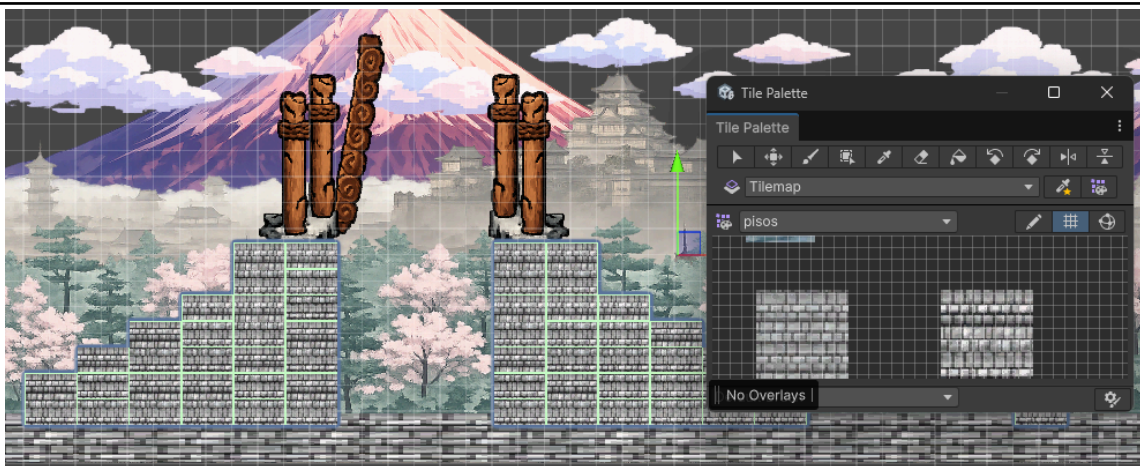
    // 2. Lógica de reposicionamiento (Looper)
    // Si la traslación acumulada supera el ancho del sprite, se aplica
una
    // traslación de corrección para reiniciar el ciclo visual.
    if(moveAmount > startPosition + spriteWidth) {
        transform.Translate(new Vector3(spriteWidth, 0, 0));
        startPosition += spriteWidth;
    }
}
```



Imgs. 09 - Fondos usado y su multiplierParallax como su orden de vista: Monte fuji con 0.9 y orden 1, Nubes con 0.7 y orden 2, Ciudad feudal con 0.5 y orden 3 y por último bosque con 0.3 y orden 4

ii. Tilemaps y Grid

Para la construcción del escenario se utilizó la técnica *Tile-Based*, que organiza los elementos gráficos en una matriz de celdas uniformes. Esto permite que la traslación del personaje Kale sea consistente con las colisiones del terreno, ya que todos los bloques están alineados en un sistema de coordenadas entero.



Img. 10 - Uso del tile para dibujar el terreno de juego además debemos asignarle su Tilemap Collider 2D, su rigidbody para físicas y por ultimo para q las mallas se adecuen su CompositeCollider

CONCLUSIONES

El uso de fórmulas matemáticas para el movimiento y el cambio de tamaño permitió que Kale se mueva de forma fluida y reaccione correctamente al entorno. Al programar el giro del personaje como un espejo y ajustar los sensores de suelo según su forma de humano o lobo se garantizó que el control fuera exacto y sin errores en los saltos o cambios de estado.

La implementación de técnicas como el Parallax y los Tilemaps facilitó la creación de un mundo con profundidad y detalle sin sobrecargar el sistema. Al mover las capas de fondo a diferentes velocidades y organizar el escenario en una rejilla se logró un nivel visualmente atractivo que funciona de manera ligera y eficiente en el motor de juego.

Se determinó que usar transformaciones matemáticas para mover objetos del entorno es mucho más confiable que depender únicamente de la física del motor. Al definir puntos exactos para el movimiento de las puertas y rotar el puente sobre un punto fijo se aseguró que

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

los obstáculos funcionen siempre de la misma manera evitando que los elementos se traben o se muevan de forma impredecible.

METODOLOGÍA DE TRABAJO

REFERENCIAS Y BIBLIOGRAFÍA

- [1] Gordon, V. S., & Clevenger, J. L. (2020). *Computer Graphics Programming in OpenGL with C++*.
[2] D. P. Kothari, G. Awari, D. Shrimankar, A. Bhende (2019), *Mathematics for Computer Graphics and Game Programming*.